

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital — Carrera de Ingeniería de Software

FORM-TL-03 — Taller

Diseño y Documentación de API REST con OpenAPI 3.0

Desarrollo paso a paso resuelto — Caso de ejemplo BiblioTech

Asignatura:	Aplicaciones Distribuidas (ISR-701) — Séptimo semestre
Período académico:	Regular 2025–2026 SPA
Modalidad:	Grupal (equipos de 3–4 estudiantes)
Carácter:	Formativa (preparatoria para PE-U3 y PFC Entrega 2)
Herramientas:	Swagger Editor (editor.swagger.io), Postman 10.x, Git/-GitHub
Entregables:	/docs/api/openapi.yaml + /docs/api/coleccion-postman.json en el repositorio GitHub del equipo

Índice

1. Propósito del taller y resultado esperado	2
2. Caso de ejemplo: BiblioTech	2
3. Paso 0 — Infraestructura desde cero: base de datos, backend Java y emisión del JWT	2
3.1. Arquitectura y flujo de los datos	3
3.2. Estructura final del proyecto y matriz de versiones	3
3.3. Docker desde cero y base de datos PostgreSQL	4
3.3.1. ¿Qué es Docker y por qué se usa aquí?	4
3.3.2. El archivo docker-compose.yml explicado línea por línea	5
3.3.3. El mecanismo docker-entrypoint-initdb.d: cómo entran los datos iniciales	6
3.3.4. Comandos Docker: arrancar, inspeccionar, entrar y destruir	7
3.4. Creación del proyecto Spring Boot	8
3.5. Configuración de la aplicación	10
3.6. Lombok: eliminación del código repetitivo	10
3.7. Capa de persistencia: entidades y repositorios JPA	11
3.8. Emisión y validación del token JWT	14
3.9. DTO, controladores REST y manejo uniforme de errores	17
3.10. Tabla de clases por paquete	22
3.11. Compilar, ejecutar y verificar de extremo a extremo	22
4. Paso 1 — Identificar recursos y diseñar las URI	22
5. Paso 2 — Tabla de endpoints (mínimo 5)	23
6. Paso 3 — Escribir el contrato OpenAPI 3.0 en Swagger Editor	23
7. Paso 4 — Validar el contrato	30
8. Paso 5 — Exportar y subir al repositorio GitHub	30

9. Paso 6 — Colección Postman con ejemplos	31
10. Lista de verificación de entrega	32
11. Adicional — De una API REST a una API RESTful, paso a paso	32
12. Cómo replicar este taller con el PFC propio	34

1. Propósito del taller y resultado esperado

Objetivo del taller

Diseñar y documentar el contrato de una **API REST** (Interfaz de Programación de Aplicaciones bajo el estilo arquitectónico *Representational State Transfer*) usando la especificación **OpenAPI 3.0**, validarlo en Swagger Editor, versionarlo en GitHub y construir su colección de pruebas en Postman, como preparación directa para la Entrega 2 del Proyecto Fin de Curso (PFC).

Este documento resuelve el taller **completo, paso a paso**, sobre un dominio **distinto al de los cuatro PFC del curso** (comercio electrónico con asistente de inteligencia artificial; sistema de gestión académica escolar; control y administración de laboratorios informáticos; soporte técnico de un proveedor de Internet). El dominio elegido es la **gestión de una biblioteca universitaria (BiblioTech)**: catálogo de libros, préstamos y devoluciones. El estudiante debe **replicar exactamente la misma secuencia de pasos sustituyendo los recursos de BiblioTech por los recursos de su propio PFC** (la Sección 12 indica el mapeo sugerido por equipo).

2. Caso de ejemplo: BiblioTech

Contexto. La biblioteca de una universidad necesita exponer un microservicio que permita: (a) consultar el catálogo de libros con paginación y filtros, (b) registrar nuevos libros, (c) eliminar libros, (d) crear préstamos a socios y (e) registrar devoluciones. Toda operación, salvo el inicio de sesión, exige un token **JWT** (*JSON Web Token*) enviado en el encabezado **Authorization: Bearer <token>**.

Regla de oro antes de escribir YAML

El contrato OpenAPI se diseña **antes** de programar (enfoque *contract-first* o “primero el contrato”). El contrato es el acuerdo entre el equipo de *backend*, el de *frontend* y el docente evaluador: si el contrato está bien definido, ambos lados pueden trabajar en paralelo.

3. Paso 0 — Infraestructura desde cero: base de datos, backend Java y emisión del JWT

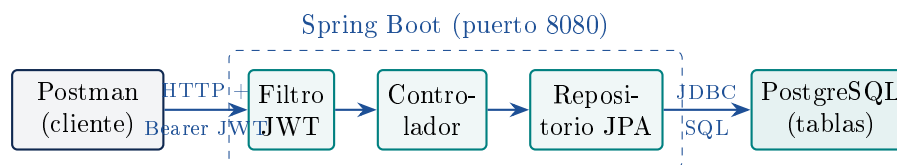
Aclaración metodológica importante

El diseño del contrato OpenAPI (Pasos 1 a 5) es **contract-first**: se puede completar **sin ninguna línea de código**, porque el contrato es un documento de diseño. Sin embargo, para que la colección Postman del Paso 6 contenga **respuestas reales** (token JWT auténtico, datos extraídos de una base de datos, códigos de estado verdaderos) y para responder con evidencia la pregunta *¿de dónde se extraen y a dónde se guardan los datos?*, este Paso 0 construye desde cero la **infraestructura mínima ejecutable**: una base de datos **PostgreSQL 18** en Docker y un **backend Java 25 (LTS) + Spring Boot 4.0.x** que implementa los endpoints y emite

el token JWT. Esta misma infraestructura es la semilla del **resource-service** que el equipo ampliará en GA-SUM-02 / PFC Entrega 2.

3.1. Arquitectura y flujo de los datos

La respuesta a “¿de dónde salen los datos?” es: **los datos persisten en tablas de PostgreSQL**; ni Postman ni Swagger Editor almacenan nada (solo envían y muestran peticiones). El recorrido completo de una petición es:



1. **Postman** envía la petición HTTP con el encabezado **Authorization: Bearer <token>**.
2. El **filtro JWT** valida la firma y la expiración del token; si es inválido, responde 401 sin tocar la base de datos.
3. El **controlador** valida la entrada (anotaciones `jakarta.validation`) y delega en el repositorio.
4. El **repositorio JPA** (Hibernate) traduce las operaciones a SQL y las ejecuta por **JDBC** contra PostgreSQL: un **POST /libros** termina en un **INSERT INTO libros...**; un **GET /libros** en un **SELECT ... LIMIT/OFFSET**.
5. La fila resultante regresa convertida a JSON: esa es la respuesta que Postman guarda como ejemplo.

3.2. Estructura final del proyecto y matriz de versiones

Antes de construir nada conviene tener a la vista **cómo debe quedar el proyecto al terminar el Paso 0**. Esta es la estructura completa de carpetas y archivos (la carpeta `docs/api/` se llena en los Pasos 5 y 6):

Listing 1: Estructura final del proyecto `bibliotech`

```

bibliotech/
|-- docker-compose.yml          <- contenedor PostgreSQL 18
|-- init.sql                    <- esquema y datos semilla
|-- pom.xml                     <- dependencias Maven
|-- .gitignore                  <- excluye target/, .env, .idea/
|-- docs/
|   '-- api/
|       |-- openapi.yaml        <- contrato OpenAPI 3.0 (Paso 5)
|       '-- coleccion-postman.json <- coleccion Postman (Paso 6)
'-- src/
    '-- main/
        |-- java/
        |   '-- com/
        |       '-- uteq/
        |           '-- bibliotech/
        |               |-- BibliotechApplication.java
        |               |-- controller/
        |                   |-- AuthController.java
        |                   |-- GlobalExceptionHandler.java
        |                   |-- LibroController.java
        |                   |-- PrestamoController.java
        |                   |-- RecursoNoEncontradoException.java
        |                   '-- ReglaNegocioException.java
        |               |-- dto/
        |                   |-- ApiError.java
        |                   |-- LibroInput.java
  
```

```

|-- LoginRequest.java
|-- PaginaResponse.java
|-- PrestamoInput.java
'-- TokenResponse.java
-- model/
|-- EstadoPrestamo.java
|-- Libro.java
|-- Prestamo.java
|-- Socio.java
'-- Usuario.java
-- repository/
|-- LibroRepository.java
|-- PrestamoRepository.java
|-- SocioRepository.java
'-- UsuarioRepository.java
'-- security/
|-- JwtAuthFilter.java
|-- JwtService.java
'-- SecurityConfig.java
'-- resources/
'-- application.yml          <- conexion a BD y parametros JWT

```

Las versiones empleadas son las **últimas estables y verificadas como compatibles entre sí** a la fecha de este documento:

Cuadro 1: Matriz de versiones compatibles del proyecto

Tecnología	Versión	Nota de compatibilidad
Java (JDK)	25 (LTS)	Soporte a largo plazo; requiere las versiones indicadas abajo
Spring Boot	4.0.x	Soporte de primera clase para Java 25; basado en Spring Framework 7
Spring Security	7.x	Incluido y gestionado por Spring Boot 4.0.x
Lombok	$\geq 1.18.42$	Primera versión estable con soporte de JDK 25
JWT	0.13.0	Última versión en Maven Central; misma API fluida de la línea 0.12
PostgreSQL	18	Imagen Docker <code>postgres:18-alpine</code>
Maven	3.9+	Requerido por Spring Boot 4.0.x

Cambio de nombre de starter en Spring Boot 4

Spring Boot 4 modularizó sus starters: el clásico `spring-boot-starter-web` se renombró a `spring-boot-starter-webmvc`. Si el equipo copia un `pom.xml` antiguo de Boot 3.x con el nombre viejo, el proyecto seguirá funcionando temporalmente (los starters clásicos quedaron como puente **obsoleto**), pero debe usarse el nombre nuevo.

3.3. Docker desde cero y base de datos PostgreSQL

3.3.1. ¿Qué es Docker y por qué se usa aquí?

Docker es una plataforma de **contenerización**: permite ejecutar una aplicación (en este caso, el servidor PostgreSQL 18) dentro de un **contenedor**, que es un proceso **aislado** que empaqueta el programa junto con todas sus dependencias, bibliotecas y configuración. A diferencia de una máquina virtual, el contenedor **no incluye un sistema operativo completo**: comparte el núcleo (kernel) del sistema operativo anfitrión, por lo que arranca en segundos y consume pocos recursos.

La distinción central es entre **imagen** y **contenedor**, análoga a la de *clase* y *objeto* en programación orientada a objetos: la **imagen** es la plantilla inmutable, de solo lectura, que contiene

PostgreSQL ya instalado y configurado (se descarga del registro público **Docker Hub**); el **contenedor** es una instancia viva creada a partir de esa imagen. De una misma imagen pueden crearse tantos contenedores como se desee.

¿Por qué usar Docker aquí en lugar de instalar PostgreSQL directamente en Windows o Linux?

1. **Entorno idéntico para los 3–4 integrantes del equipo:** todos ejecutan exactamente la misma versión de PostgreSQL con la misma configuración, sin importar su sistema operativo. Desaparece el clásico “en mi máquina sí funciona”.
2. **Sin instalación ni contaminación del sistema:** no se toca el registro de Windows ni los servicios del sistema; eliminar el contenedor deja la computadora exactamente como estaba.
3. **Arranque y reinicio reproducibles:** el archivo `docker-compose.yml` versionado en Git es la documentación ejecutable del entorno; cualquier integrante clona el repositorio y levanta la base con un solo comando.
4. **Continuidad con el PFC:** en la Entrega 2 (GA-SUM-02) el equipo contenerizará también los microservicios con Dockerfile y los orquestará con este mismo Docker Compose; lo aprendido aquí se reutiliza íntegro.

Cuadro 2: Conceptos fundamentales de Docker y su materialización en este proyecto

Concepto	Definición	En este proyecto
Imagen	Plantilla inmutable con la aplicación y sus dependencias	<code>postgres:18-alpine</code>
Contenedor	Instancia en ejecución de una imagen, aislada del anfitrión	<code>biotech-db</code>
Registro	Repositorio remoto de imágenes	Docker Hub (descarga automática)
Puerto publicado	Túnel que conecta un puerto de la PC con uno del contenedor	<code>5432:5432</code>
Bind mount	Monta un archivo/carpetas de la PC dentro del contenedor	<code>./init.sql</code>
Volumen nombrado	Almacenamiento persistente administrado por Docker	<code>datos-pg</code>
Docker Compose	Herramienta que define y levanta servicios desde un YAML	<code>docker-compose.yml</code>

Requisito previo: tener instalado Docker Desktop 4.x (Windows/macOS) o Docker Engine con el plugin Compose (Linux). Verificar la instalación:

```
docker --version           # muestra la version del motor Docker instalado
docker compose version     # confirma que el plugin Compose esta disponible
docker run hello-world     # descarga y ejecuta una imagen de prueba minima:
                           # si imprime un saludo, Docker funciona correctamente
```

3.3.2. El archivo `docker-compose.yml` explicado línea por línea

Docker Compose lee este archivo YAML (sangría de 2 espacios, obligatoria) y a partir de él descarga la imagen, crea el contenedor, publica el puerto y monta los volúmenes. Crear el archivo en la **raíz del proyecto** con este contenido; cada línea está comentada:

Listing 2: `docker-compose.yml` — comentado línea por línea

```
1 services:                                # Seccion raiz: lista de contenedores (servicios)
2                                           # que componen el sistema; aqui solo uno.
3
4     db:                                  # Nombre logico del servicio dentro de Compose.
5                                           # Otros contenedores de la misma red lo
6                                           # alcanzarian usando "db" como nombre de host.
7
8     image: postgres:18-alpine           # Imagen a usar: repositorio "postgres",
```

```

9          # etiqueta "18-alpine" = PostgreSQL version 18
10         # sobre Alpine Linux (variante minima, ~90 MB).
11         # Si no existe en la PC, Docker la descarga
12         # automaticamente desde Docker Hub.
13
14     container_name: bibliotech-db    # Nombre fijo del contenedor, usado en
15                                     # comandos como "docker exec bibliotech-db".
16                                     # Sin esta linea, Compose generaria un
17                                     # nombre automatico (p. ej. tl03-db-1).
18
19     environment:                    # Variables de entorno que la imagen oficial
20                                     # de PostgreSQL lee en su PRIMER arranque
21                                     # para autoconfigurarse:
22     POSTGRES_DB: bibliotech         # crea automaticamente la base "bibliotech"
23     POSTGRES_USER: biblio           # crea el usuario administrador "biblio"
24     POSTGRES_PASSWORD: biblio123    # contrasena de ese usuario (obligatoria;
25                                     # sin ella el contenedor se niega a arrancar)
26
27     ports:                          # Publicacion de puertos, formato "PC:contenedor"
28     - "5432:5432"                  # Conecta el puerto 5432 de la computadora con
29                                     # el 5432 interno donde escucha PostgreSQL.
30                                     # Gracias a esto, Spring Boot (que corre FUERA
31                                     # del contenedor) se conecta via
32                                     # jdbc:postgresql://localhost:5432/bibliotech
33
34     volumes:                        # Montajes de almacenamiento del contenedor:
35     - ./init.sql:/docker-entrypoint-initdb.d/init.sql
36                                     # BIND MOUNT: monta el archivo local ./init.sql
37                                     # dentro del contenedor, en la carpeta especial
38                                     # /docker-entrypoint-initdb.d/. Todo script .sql
39                                     # colocado alli se ejecuta automaticamente en el
40                                     # primer arranque (ver mecanismo mas abajo).
41     - datos-pg:/var/lib/postgresql/data
42                                     # VOLUMEN NOMBRADO: /var/lib/postgresql/data es
43                                     # el directorio donde PostgreSQL escribe los
44                                     # datos reales (tablas, indices, WAL). Al
45                                     # montarlo en el volumen "datos-pg", los datos
46                                     # SOBREVIVEN aunque el contenedor se detenga,
47                                     # se elimine o se recree.
48
49     volumes:                        # Declaracion de los volúmenes nombrados
50     datos-pg:                      # Docker crea y administra este volumen en su
51                                     # area interna; se borra solo con "down -v".

```

3.3.3. El mecanismo docker-entrypoint-initdb.d: cómo entran los datos iniciales

La imagen oficial de PostgreSQL incluye un guion de arranque (*entrypoint*) que, **solo cuando el volumen de datos está vacío** (es decir, en el primer arranque del servicio), ejecuta en orden alfabético todos los archivos `.sql` y `.sh` que encuentre en la carpeta `/docker-entrypoint-initdb.d/`. Como el bind mount colocó allí nuestro `init.sql`, ese script crea el esquema (tablas usuarios, socios, libros, prestamos) e inserta los datos semilla **sin intervención manual**. En arranques posteriores, el volumen ya contiene datos y el script **se ignora por completo**: por eso editar `init.sql` no surte efecto hasta destruir el volumen (ver ciclo de vida más abajo).

El contenido de `init.sql` es el siguiente:

Listing 3: `init.sql` — esquema y datos semilla (aquí se guardan los datos)

```

1  -- ===== ESQUEMA =====
2  CREATE TABLE usuarios (
3      id                BIGSERIAL PRIMARY KEY,
4      email             VARCHAR(120) NOT NULL UNIQUE,
5      password_hash     VARCHAR(72)  NOT NULL,      -- hash BCrypt, nunca texto plano
6      nombre            VARCHAR(120) NOT NULL

```

```

7 );
8
9 CREATE TABLE socios (
10     id          BIGSERIAL PRIMARY KEY,
11     nombre      VARCHAR(120) NOT NULL,
12     email       VARCHAR(120) NOT NULL UNIQUE
13 );
14
15 CREATE TABLE libros (
16     id          BIGSERIAL PRIMARY KEY,
17     titulo      VARCHAR(200) NOT NULL,
18     autor       VARCHAR(120) NOT NULL,
19     isbn        VARCHAR(17)  NOT NULL UNIQUE,
20     categoria   VARCHAR(80)  NOT NULL,
21     ejemplares_disponibles INTEGER NOT NULL CHECK (ejemplares_disponibles >= 0)
22 );
23
24 CREATE TABLE prestamos (
25     id          BIGSERIAL PRIMARY KEY,
26     libro_id    BIGINT NOT NULL REFERENCES libros(id),
27     socio_id    BIGINT NOT NULL REFERENCES socios(id),
28     fecha_prestamo DATE NOT NULL DEFAULT CURRENT_DATE,
29     fecha_limite DATE NOT NULL,
30     estado      VARCHAR(10) NOT NULL DEFAULT 'ACTIVO'
31               CHECK (estado IN ('ACTIVO', 'DEVUELTO', 'VENCIDO'))
32 );
33
34 -- ===== DATOS SEMILLA =====
35 -- Password del usuario demo: Secreto123! (hash BCrypt real, costo 10)
36 INSERT INTO usuarios (email, password_hash, nombre) VALUES
37 ('maria.perez@uteq.edu.ec',
38  '$2b$10$v3o52i/.9U8c9MPgcyRyMOFmTEjR43AeoWzbvg0vLAIdMO6Z5oTWq',
39  'Maria Perez');
40
41 INSERT INTO socios (nombre, email) VALUES
42 ('Carlos Vera', 'carlos.vera@uteq.edu.ec'),
43 ('Ana Sanchez', 'ana.sanchez@uteq.edu.ec');
44
45 INSERT INTO libros (titulo, autor, isbn, categoria, ejemplares_disponibles) VALUES
46 ('Building Microservices', 'Sam Newman', '978-1-4920-3402-5', '
47   Ingenieria de Software', 4),
48 ('Patterns of Distributed Systems', 'Unmesh Joshi', '978-0-13-822198-0', '
49   Sistemas Distribuidos', 2),
50 ('Designing Data-Intensive Applications', 'Martin Kleppmann', '978-1-449-37332-0',
51   'Bases de Datos', 3);

```

3.3.4. Comandos Docker: arrancar, inspeccionar, entrar y destruir

Con los dos archivos creados, el ciclo completo de operación de la base de datos se maneja con los siguientes comandos, ejecutados desde la raíz del proyecto (donde está `docker-compose.yml`); cada línea está comentada:

Listing 4: Comandos de operación del contenedor, comentados

```

docker compose up -d db
# "up": descarga la imagen si falta, crea el contenedor, monta
#       volúmenes, publica puertos y lo inicia.
# "-d": modo "detached" (segundo plano); la terminal queda libre.
# "db": nombre del servicio definido en docker-compose.yml.

docker compose ps
# Lista los servicios de ESTE compose con su estado.
# Debe mostrar bibliotech-db con STATUS "Up" y el puerto 5432->5432.

```

```

docker compose logs -f db
# Muestra la salida interna del contenedor (consola de PostgreSQL).
# "-f" (follow) la transmite en vivo; se sale con Ctrl+C (el
# contenedor NO se detiene, solo se deja de mirar).
# En el primer arranque debe verse la ejecucion de init.sql y al
# final: "database system is ready to accept connections".

docker exec -it bibliotech-db psql -U biblio -d bibliotech
# "exec": ejecuta un comando ADENTRO de un contenedor ya corriendo.
# "-i" mantiene la entrada abierta y "-t" asigna una terminal:
#      juntas permiten una sesion interactiva.
# "bibliotech-db": nombre del contenedor (container_name).
# "psql -U biblio -d bibliotech": cliente de PostgreSQL, conectado
#      como usuario "biblio" a la base "bibliotech".
# Adentro, probar: |dt                (listar tablas)
#                  |SELECT * FROM libros;
#                  |q                (salir)

docker compose stop db
# Detiene el contenedor SIN eliminarlo (como apagar el servidor).
# Se reanuda con "docker compose start db".

docker compose down
# Detiene Y ELIMINA el contenedor y la red creada por Compose.
# El volumen "datos-pg" se conserva: los datos NO se pierden;
# el proximo "up -d" recrea el contenedor sobre los mismos datos.

docker compose down -v
# Igual que down, pero ademas BORRA los volúmenes ("-v").
# Es el "reinicio de fabrica": la proxima vez que se levante el
# servicio, el volumen estara vacio y init.sql se ejecutara de nuevo.

```

Ciclo de vida de los datos: la duda más frecuente

Los datos viven en el **volumen datos-pg**, no en el contenedor. Por eso: (a) detener o eliminar el contenedor (**stop**, **down**) **no** borra los datos; (b) **init.sql** se ejecuta **una sola vez en la vida del volumen**, así que si el equipo modifica ese script (por ejemplo, para añadir más libros semilla) debe ejecutar **docker compose down -v** y luego **docker compose up -d db** para que el cambio se aplique; y (c) los registros creados después por la API (**INSERT** vía **POST /libros**) persisten en el volumen entre reinicios, que es exactamente el comportamiento esperado de una base de datos real.

Seguridad de credenciales

Las contraseñas de los usuarios **jamás se guardan en texto plano**: la columna **password_hash** almacena un hash **BCrypt** (irreversible, con sal aleatoria). En el repositorio del PFC, las credenciales de la base de datos deben ir en un **.env** excluido por **.gitignore**, publicando solo un **.env.example** sin valores reales.

3.4. Creación del proyecto Spring Boot

Generación del esqueleto

1. Abrir <https://start.spring.io> y configurar: **Maven**, **Java 25**, Spring Boot **4.0.x** (última estable), Group **com.uteq**, Artifact **bibliotech**, empaquetado **Jar**.
2. Dependencias: **Spring Web** (en Boot 4 genera el starter **spring-boot-starter-webmvc**), **Spring Data JPA**, **PostgreSQL Driver**, **Spring Security**, **Validation** y **Lombok**.
3. Generar, descomprimir y abrir en IntelliJ IDEA 2025.x. Verificar que **Annotation Proces-**

- `sing` esté habilitado (Settings → Build → Compiler → Annotation Processors → Enable), requisito para que Lombok genere el código en la compilación.
- Añadir manualmente al `pom.xml` la biblioteca **JWT** (creación y validación de JWT), que no está en el `Initializr`.

Listing 5: `pom.xml` — propiedades, dependencias y compilador (Java 25 + Boot 4.0.x)

```

1 <parent>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-parent</artifactId>
4   <version>4.0.6</version>
5   <relativePath/>
6 </parent>
7
8 <properties>
9   <java.version>25</java.version>
10 </properties>
11
12 <dependencies>
13   <!-- En Spring Boot 4, el starter web MVC se llama webmvc -->
14   <dependency>
15     <groupId>org.springframework.boot</groupId>
16     <artifactId>spring-boot-starter-webmvc</artifactId>
17   </dependency>
18   <dependency>
19     <groupId>org.springframework.boot</groupId>
20     <artifactId>spring-boot-starter-data-jpa</artifactId>
21   </dependency>
22   <dependency>
23     <groupId>org.springframework.boot</groupId>
24     <artifactId>spring-boot-starter-security</artifactId>
25   </dependency>
26   <dependency>
27     <groupId>org.springframework.boot</groupId>
28     <artifactId>spring-boot-starter-validation</artifactId>
29   </dependency>
30   <dependency>
31     <groupId>org.postgresql</groupId>
32     <artifactId>postgresql</artifactId>
33     <scope>runtime</scope>
34   </dependency>
35   <!-- Lombok: version gestionada por el parent (>= 1.18.42 para JDK 25) -->
36   <dependency>
37     <groupId>org.projectlombok</groupId>
38     <artifactId>lombok</artifactId>
39     <optional>true</optional>
40   </dependency>
41   <!-- JWT: emision y verificacion de tokens JWT -->
42   <dependency>
43     <groupId>io.jsonwebtoken</groupId>
44     <artifactId>jjwt-api</artifactId>
45     <version>0.13.0</version>
46   </dependency>
47   <dependency>
48     <groupId>io.jsonwebtoken</groupId>
49     <artifactId>jjwt-impl</artifactId>
50     <version>0.13.0</version>
51     <scope>runtime</scope>
52   </dependency>
53   <dependency>
54     <groupId>io.jsonwebtoken</groupId>
55     <artifactId>jjwt-jackson</artifactId>
56     <version>0.13.0</version>

```

```

57     <scope>runtime</scope>
58   </dependency>
59 </dependencies>
60
61 <build>
62   <plugins>
63     <plugin>
64       <groupId>org.springframework.boot</groupId>
65       <artifactId>spring-boot-maven-plugin</artifactId>
66       <configuration>
67         <excludes>
68           <exclude>
69             <groupId>org.projectlombok</groupId>
70             <artifactId>lombok</artifactId>
71           </exclude>
72         </excludes>
73       </configuration>
74     </plugin>
75     <!-- Desde JDK 23 el compilador desactiva por defecto los procesadores
76          de anotaciones (-proc:none); hay que declarar Lombok explícitamente -->
77     <plugin>
78       <groupId>org.apache.maven.plugins</groupId>
79       <artifactId>maven-compiler-plugin</artifactId>
80       <configuration>
81         <annotationProcessorPaths>
82           <path>
83             <groupId>org.projectlombok</groupId>
84             <artifactId>lombok</artifactId>
85           </path>
86         </annotationProcessorPaths>
87       </configuration>
88     </plugin>
89   </plugins>
90 </build>

```

3.5. Configuración de la aplicación

Listing 6: src/main/resources/application.yml — conexión a PostgreSQL y parámetros JWT

```

1 server:
2   port: 8080
3
4 spring:
5   datasource:
6     url: jdbc:postgresql://localhost:5432/bibliotech
7     username: biblio
8     password: biblio123
9   jpa:
10    hibernate:
11      ddl-auto: validate    # el esquema lo creo init.sql; Hibernate solo valida
12    open-in-view: false
13
14 bibliotech:
15   jwt:
16     # HS256 exige una clave de al menos 32 bytes. En produccion: variable de
17     entorno.
18     secreto: CambiarEsteSecretoEnProduccion-UTEQ-AD-2026-Minimo32Bytes!!
19     expiracion-segundos: 3600

```

3.6. Lombok: eliminación del código repetitivo

Antes de escribir las clases Java conviene entender **Lombok**, biblioteca que se usará en todo el código siguiente. Lombok es un **procesador de anotaciones**: actúa durante la compilación, leyendo

anotaciones como `@Getter` y **generando dentro del bytecode** el código repetitivo (*boilerplate*) que de otro modo habría que escribir a mano. Sus ventajas principales:

1. **Menos código repetitivo.** Getters, setters, constructores, `equals()`, `hashCode()` y `toString()` se generan automáticamente; la entidad `Libro`, por ejemplo, pasa de unas 45 líneas a unas 25 sin perder ninguna funcionalidad.
2. **Cero costo en tiempo de ejecución.** Todo ocurre al compilar: el `.class` resultante es idéntico al que se obtendría escribiendo el código a mano; no hay reflexión ni proxies en producción.
3. **Mayor legibilidad y mantenibilidad.** La clase muestra solo lo esencial (campos y lógica de negocio); al agregar un campo nuevo no hay que recordar crear su getter ni actualizar constructores.
4. **Menos errores.** Elimina los descuidos típicos del código manual: getters que devuelven el campo equivocado, constructores desactualizados o `equals()` inconsistentes con `hashCode()`.
5. **Inyección de dependencias limpia.** `@RequiredArgsConstructor` genera el constructor con todos los campos `final`, que es exactamente lo que Spring necesita para la inyección por constructor (la forma recomendada); los controladores quedan sin constructores escritos a mano.

Las anotaciones que se usarán y su efecto:

Cuadro 3: Anotaciones Lombok empleadas en el proyecto

Anotación	Código que genera
<code>@Getter</code>	Un getter por campo (a nivel de clase) o para un campo concreto
<code>@Setter</code>	Un setter; aquí se aplica solo a campos específicos que deben mutar
<code>@NoArgsConstructor</code>	Constructor sin argumentos; con <code>access = AccessLevel.PROTECTED</code> satisface el requisito de JPA sin exponerlo públicamente
<code>@RequiredArgsConstructor</code>	Constructor con todos los campos <code>final</code> (inyección por constructor en Spring)

Dos precauciones de uso profesional

- (1) **No usar `@Data` en entidades JPA:** genera `equals()`, `hashCode()` y `toString()` sobre todos los campos, lo que provoca problemas con identidades de entidad, colecciones y relaciones de carga perezosa; en entidades se prefieren `@Getter`/`@Setter` granulares, como se hace aquí.
- (2) **Los record de Java no necesitan Lombok:** el propio lenguaje ya genera constructor, accesores, `equals()`, `hashCode()` y `toString()`; por eso los DTO de este proyecto siguen siendo records sin anotaciones Lombok.

3.7. Capa de persistencia: entidades y repositorios JPA

Cada tabla de `init.sql` se refleja en una **entidad** Java; los **repositorios** heredan de `JpaRepository` y obtienen gratis las operaciones CRUD (crear, leer, actualizar, eliminar) y la paginación.

Listing 7: `model/Libro.java` — entidad mapeada a la tabla `libros` (con Lombok)

```

1 package com.uteq.bibliotech.model;
2
3 import jakarta.persistence.*;
4 import lombok.AccessLevel;
5 import lombok.Getter;
6 import lombok.NoArgsConstructor;
7 import lombok.Setter;
8
9 @Entity
10 @Table(name = "libros")
11 @Getter

```

```

12 @NoArgsConstructor(access = AccessLevel.PROTECTED)    // constructor vacio para JPA
13 public class Libro {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @Column(nullable = false, length = 200)
20     private String titulo;
21
22     @Column(nullable = false, length = 120)
23     private String autor;
24
25     @Column(nullable = false, unique = true, length = 17)
26     private String isbn;
27
28     @Column(nullable = false, length = 80)
29     private String categoria;
30
31     @Setter    // unico campo que muta tras la creacion (prestamos/devoluciones)
32     @Column(name = "ejemplares_disponibles", nullable = false)
33     private Integer ejemplaresDisponibles;
34
35     public Libro(String titulo, String autor, String isbn,
36                  String categoria, Integer ejemplaresDisponibles) {
37         this.titulo = titulo;
38         this.autor = autor;
39         this.isbn = isbn;
40         this.categoria = categoria;
41         this.ejemplaresDisponibles = ejemplaresDisponibles;
42     }
43 }

```

Listing 8: model/Prestamo.java, model/Socio.java y model/Usuario.java (con Lombok)

```

1 // ===== archivo: model/EstadoPrestamo.java =====
2 package com.uteq.bibliotech.model;
3
4 public enum EstadoPrestamo { ACTIVO, DEVUELTO, VENCIDO }
5
6 // ===== archivo: model/Prestamo.java =====
7 package com.uteq.bibliotech.model;
8
9 import jakarta.persistence.*;
10 import lombok.AccessLevel;
11 import lombok.Getter;
12 import lombok.NoArgsConstructor;
13 import lombok.Setter;
14 import java.time.LocalDate;
15
16 @Entity
17 @Table(name = "prestamos")
18 @Getter
19 @NoArgsConstructor(access = AccessLevel.PROTECTED)
20 public class Prestamo {
21
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     private Long id;
25
26     @Column(name = "libro_id", nullable = false)
27     private Long libroId;
28
29     @Column(name = "socio_id", nullable = false)
30     private Long socioId;

```

```

31
32     @Column(name = "fecha_prestamo", nullable = false)
33     private LocalDate fechaPrestamo;
34
35     @Column(name = "fecha_limite", nullable = false)
36     private LocalDate fechaLimite;
37
38     @Setter // unico campo mutable: ACTIVO -> DEVUELTO
39     @Enumerated(EnumType.STRING)
40     @Column(nullable = false, length = 10)
41     private EstadoPrestamo estado;
42
43     public Prestamo(Long libroId, Long socioId, LocalDate fechaLimite) {
44         this.libroId = libroId;
45         this.socioId = socioId;
46         this.fechaPrestamo = LocalDate.now();
47         this.fechaLimite = fechaLimite;
48         this.estado = EstadoPrestamo.ACTIVO;
49     }
50 }
51
52 // ===== archivo: model/Socio.java =====
53 package com.uteq.bibliotech.model;
54
55 import jakarta.persistence.*;
56 import lombok.AccessLevel;
57 import lombok.Getter;
58 import lombok.NoArgsConstructor;
59
60 @Entity
61 @Table(name = "socios")
62 @Getter
63 @NoArgsConstructor(access = AccessLevel.PROTECTED)
64 public class Socio {
65
66     @Id
67     @GeneratedValue(strategy = GenerationType.IDENTITY)
68     private Long id;
69
70     @Column(nullable = false, length = 120)
71     private String nombre;
72
73     @Column(nullable = false, unique = true, length = 120)
74     private String email;
75 }
76
77 // ===== archivo: model/Usuario.java =====
78 package com.uteq.bibliotech.model;
79
80 import jakarta.persistence.*;
81 import lombok.AccessLevel;
82 import lombok.Getter;
83 import lombok.NoArgsConstructor;
84
85 @Entity
86 @Table(name = "usuarios")
87 @Getter
88 @NoArgsConstructor(access = AccessLevel.PROTECTED)
89 public class Usuario {
90
91     @Id
92     @GeneratedValue(strategy = GenerationType.IDENTITY)
93     private Long id;
94
95     @Column(nullable = false, unique = true, length = 120)

```

```

96     private String email;
97
98     @Column(name = "password_hash", nullable = false, length = 72)
99     private String passwordHash;
100
101     @Column(nullable = false, length = 120)
102     private String nombre;
103 }

```

Listing 9: repository/ — repositorios JPA (de aquí “se extraen” los datos)

```

1 // ===== archivo: repository/LibroRepository.java =====
2 package com.uteq.bibliotech.repository;
3
4 import com.uteq.bibliotech.model.Libro;
5 import org.springframework.data.domain.Page;
6 import org.springframework.data.domain.Pageable;
7 import org.springframework.data.jpa.repository.JpaRepository;
8
9 public interface LibroRepository extends JpaRepository<Libro, Long> {
10     Page<Libro> findByCategoriaIgnoreCase(String categoria, Pageable pageable);
11 }
12
13 // ===== archivo: repository/PrestamoRepository.java =====
14 package com.uteq.bibliotech.repository;
15
16 import com.uteq.bibliotech.model.Prestamo;
17 import org.springframework.data.jpa.repository.JpaRepository;
18
19 public interface PrestamoRepository extends JpaRepository<Prestamo, Long> { }
20
21 // ===== archivo: repository/SocioRepository.java =====
22 package com.uteq.bibliotech.repository;
23
24 import com.uteq.bibliotech.model.Socio;
25 import org.springframework.data.jpa.repository.JpaRepository;
26
27 public interface SocioRepository extends JpaRepository<Socio, Long> { }
28
29 // ===== archivo: repository/UsuarioRepository.java =====
30 package com.uteq.bibliotech.repository;
31
32 import com.uteq.bibliotech.model.Usuario;
33 import org.springframework.data.jpa.repository.JpaRepository;
34 import java.util.Optional;
35
36 public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
37     Optional<Usuario> findByEmail(String email);
38 }

```

3.8. Emisión y validación del token JWT

Un JWT tiene tres partes codificadas en Base64URL y separadas por puntos: `header.payload.signature`. El *header* declara el algoritmo (HS256); el *payload* lleva los *claims* (`sub` = email del usuario, `iat` = emisión, `exp` = expiración); la *firma* es un HMAC-SHA256 del header y el payload con la clave secreta del servidor: si alguien altera el contenido, la firma deja de coincidir y el token se rechaza. Esto hace posible la restricción **stateless**: el servidor no guarda sesiones porque toda la identidad viaja firmada en cada petición.

Listing 10: security/JwtService.java — crea y valida el token (aquí “nace” el JWT)

```

1 package com.uteq.bibliotech.security;
2

```

```

3 import io.jsonwebtoken.Claims;
4 import io.jsonwebtoken.Jwts;
5 import io.jsonwebtoken.security.Keys;
6 import lombok.Getter;
7 import org.springframework.beans.factory.annotation.Value;
8 import org.springframework.stereotype.Service;
9
10 import javax.crypto.SecretKey;
11 import java.nio.charset.StandardCharsets;
12 import java.util.Date;
13
14 @Service
15 public class JwtService {
16
17     private final SecretKey clave;
18
19     @Getter // lo consultara AuthController para armar la respuesta
20     private final long expiracionSegundos;
21
22     public JwtService(
23         @Value("${bibliotech.jwt.secreto}") String secreto,
24         @Value("${bibliotech.jwt.expiracion-segundos}") long expiracionSegundos
25     ) {
26         this.clave = Keys.hmacShaKeyFor(secreto.getBytes(StandardCharsets.UTF_8));
27         this.expiracionSegundos = expiracionSegundos;
28     }
29
30     /** Emite un JWT firmado HS256 con expiracion configurada. */
31     public String generarToken(String email) {
32         Date ahora = new Date();
33         Date expira = new Date(ahora.getTime() + expiracionSegundos * 1000);
34         return Jwts.builder()
35             .subject(email) // claim "sub"
36             .issuedAt(ahora) // claim "iat"
37             .expiration(expira) // claim "exp"
38             .signWith(clave) // firma HMAC-SHA256
39             .compact();
40     }
41
42     /** Verifica firma y expiracion; retorna el email (claim sub).
43     Lanza io.jsonwebtoken.JwtException si el token es invalido. */
44     public String validarYExtraerEmail(String token) {
45         Claims claims = Jwts.parser()
46             .verifyWith(clave)
47             .build()
48             .parseSignedClaims(token)
49             .getPayload();
50         return claims.getSubject();
51     }
52 }

```

Listing 11: security/JwtAuthFilter.java y security/SecurityConfig.java

```

1 // ===== archivo: security/JwtAuthFilter.java =====
2 package com.uteq.bibliotech.security;
3
4 import io.jsonwebtoken.JwtException;
5 import jakarta.servlet.FilterChain;
6 import jakarta.servlet.ServletException;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import lombok.RequiredArgsConstructor;
10 import org.springframework.security.authentication.
    UsernamePasswordAuthenticationToken;
11 import org.springframework.security.core.context.SecurityContextHolder;

```

```

12 import org.springframework.stereotype.Component;
13 import org.springframework.web.filter.OncePerRequestFilter;
14
15 import java.io.IOException;
16 import java.util.List;
17
18 @Component
19 @RequiredArgsConstructor // genera el constructor con el campo final (inyeccion)
20 public class JwtAuthFilter extends OncePerRequestFilter {
21
22     private final JwtService jwtService;
23
24     @Override
25     protected void doFilterInternal(HttpServletRequest request,
26                                     HttpServletResponse response,
27                                     FilterChain chain)
28         throws ServletException, IOException {
29         String header = request.getHeader("Authorization");
30         if (header != null && header.startsWith("Bearer ")) {
31             try {
32                 String email = jwtService.validarYExtraerEmail(header.substring(7))
33                     ;
34                 var auth = new UsernamePasswordAuthenticationToken(email, null,
35                             List.of());
36                 SecurityContextHolder.getContext().setAuthentication(auth);
37             } catch (JwtException e) {
38                 SecurityContextHolder.clearContext(); // token invalido ->
39                 respondera 401
40             }
41         }
42         chain.doFilter(request, response);
43     }
44 }
45
46 // ==== archivo: security/SecurityConfig.java ====
47 package com.uteq.bibliotech.security;
48
49 import org.springframework.context.annotation.Bean;
50 import org.springframework.context.annotation.Configuration;
51 import org.springframework.http.HttpStatus;
52 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
53 import org.springframework.security.config.http.SessionCreationPolicy;
54 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
55 import org.springframework.security.crypto.password.PasswordEncoder;
56 import org.springframework.security.web.SecurityFilterChain;
57 import org.springframework.security.web.authentication.HttpStatusEntryPoint;
58 import org.springframework.security.web.authentication.
59     UsernamePasswordAuthenticationFilter;
60
61 @Configuration
62 public class SecurityConfig {
63
64     @Bean
65     public SecurityFilterChain filterChain(HttpSecurity http, JwtAuthFilter
66         jwtFilter)
67         throws Exception {
68         return http
69             .csrf(csrf -> csrf.disable()) // API stateless:
70             .sin CSRF
71             .sessionManagement(s ->
72                 s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
73             .authorizeHttpRequests(auth -> auth
74                 .requestMatchers("/api/v1/auth/**").permitAll() // login es publico
75                 .anyRequest().authenticated() // todo lo demas
76                 exige JWT
77             )
78         ;
79     }
80 }

```



```

70         .exceptionHandling(e -> e.authenticationEntryPoint(
71             new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))) // 401 sin
              token
72         .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
73         .build();
74     }
75
76     @Bean
77     public PasswordEncoder passwordEncoder() {
78         return new BCryptPasswordEncoder();
79     }
80 }

```

3.9. DTO, controladores REST y manejo uniforme de errores

Los **DTO** (objetos de transferencia de datos, aquí **record** de Java 25) definen exactamente lo que entra y sale por la API — son la contraparte en código de los **components/schemas** del contrato OpenAPI. Nótese que los records **no llevan anotaciones Lombok**: el lenguaje ya les genera constructor, accesores, `equals()`, `hashCode()` y `toString()`.

Listing 12: `dto/` — records de entrada/salida (espejo de `components/schemas`)

```

1 // ===== archivo: dto/LoginRequest.java =====
2 package com.uteq.bibliotech.dto;
3
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6 import jakarta.validation.constraints.Size;
7
8 public record LoginRequest(
9     @NotBlank @Email String email,
10    @NotBlank @Size(min = 8) String password) { }
11
12 // ===== archivo: dto/TokenResponse.java =====
13 package com.uteq.bibliotech.dto;
14
15 public record TokenResponse(String token, String tipo, long expiraEn) { }
16
17 // ===== archivo: dto/LibroInput.java =====
18 package com.uteq.bibliotech.dto;
19
20 import jakarta.validation.constraints.*;
21
22 public record LibroInput(
23     @NotBlank String titulo,
24     @NotBlank String autor,
25     @NotBlank @Pattern(regexp = "[0-9-]{10,17}$") String isbn,
26     @NotBlank String categoria,
27     @NotNull @Min(0) Integer ejemplaresDisponibles) { }
28
29 // ===== archivo: dto/PrestamoInput.java =====
30 package com.uteq.bibliotech.dto;
31
32 import jakarta.validation.constraints.Future;
33 import jakarta.validation.constraints.NotNull;
34 import java.time.LocalDate;
35
36 public record PrestamoInput(
37     @NotNull Long libroId,
38     @NotNull Long socioId,
39     @NotNull @Future LocalDate fechaLimite) { }
40
41 // ===== archivo: dto/PaginaResponse.java =====
42 package com.uteq.bibliotech.dto;

```

```

43
44 import java.util.List;
45
46 public record PaginaResponse<T>(
47     List<T> contenido, int pagina, int totalPaginas, long totalElementos) { }
48
49 // ===== archivo: dto/ApiError.java =====
50 package com.uteq.bibliotech.dto;
51
52 public record ApiError(
53     int status, String error, String message,
54     String timestamp, String path) { }

```

Listing 13: controller/AuthController.java — POST /auth/login emite el JWT (con Lombok)

```

1 package com.uteq.bibliotech.controller;
2
3 import com.uteq.bibliotech.dto.LoginRequest;
4 import com.uteq.bibliotech.dto.TokenResponse;
5 import com.uteq.bibliotech.model.Usuario;
6 import com.uteq.bibliotech.repository.UsuarioRepository;
7 import com.uteq.bibliotech.security.JwtService;
8 import jakarta.validation.Valid;
9 import lombok.RequiredArgsConstructor;
10 import org.springframework.security.authentication.BadCredentialsException;
11 import org.springframework.security.crypto.password.PasswordEncoder;
12 import org.springframework.web.bind.annotation.*;
13
14 @RestController
15 @RequestMapping("/api/v1/auth")
16 @RequiredArgsConstructor // constructor con los tres campos final
17 public class AuthController {
18
19     private final UsuarioRepository usuarios;
20     private final PasswordEncoder encoder;
21     private final JwtService jwtService;
22
23     @PostMapping("/login")
24     public TokenResponse login(@Valid @RequestBody LoginRequest req) {
25         Usuario u = usuarios.findByEmail(req.email())
26             .orElseThrow(() -> new BadCredentialsException("Credenciales
27                 invalidas"));
28         if (!encoder.matches(req.password(), u.getPasswordHash())) {
29             throw new BadCredentialsException("Credenciales invalidas");
30         }
31         return new TokenResponse(jwtService.generarToken(u.getEmail()),
32             "Bearer", jwtService.getExpiracionSegundos());
33     }
34 }

```

Listing 14: controller/LibroController.java y controller/PrestamoController.java

```

1 // ===== archivo: controller/RecursoNoEncontradoException.java =====
2 package com.uteq.bibliotech.controller;
3
4 public class RecursoNoEncontradoException extends RuntimeException {
5     public RecursoNoEncontradoException(String mensaje) { super(mensaje); }
6 }
7
8 // ===== archivo: controller/ReglaNegocioException.java =====
9 package com.uteq.bibliotech.controller;
10
11 public class ReglaNegocioException extends RuntimeException {
12     public ReglaNegocioException(String mensaje) { super(mensaje); }
13 }

```

```

14
15 // ===== archivo: controller/LibroController.java =====
16 package com.uteq.bibliotech.controller;
17
18 import com.uteq.bibliotech.dto.LibroInput;
19 import com.uteq.bibliotech.dto.PaginaResponse;
20 import com.uteq.bibliotech.model.Libro;
21 import com.uteq.bibliotech.repository.LibroRepository;
22 import jakarta.validation.Valid;
23 import lombok.RequiredArgsConstructor;
24 import org.springframework.data.domain.Page;
25 import org.springframework.data.domain.PageRequest;
26 import org.springframework.http.ResponseEntity;
27 import org.springframework.web.bind.annotation.*;
28 import java.net.URI;
29
30 @RestController
31 @RequestMapping("/api/v1/libros")
32 @RequiredArgsConstructor
33 public class LibroController {
34
35     private final LibroRepository libros;
36
37     @GetMapping
38     public PaginaResponse<Libro> listar(
39         @RequestParam(defaultValue = "0") int page,
40         @RequestParam(defaultValue = "20") int size,
41         @RequestParam(required = false) String categoria) {
42         Page<Libro> pagina = (categoria == null)
43             ? libros.findAll(PageRequest.of(page, size))
44             : libros.findByCategoriaIgnoreCase(categoria, PageRequest.of(page,
45                 size));
46         return new PaginaResponse<>(pagina.getContent(), pagina.getNumber(),
47             pagina.getTotalPages(), pagina.getTotalElements());
48     }
49
50     @GetMapping("/{libroId}")
51     public Libro obtener(@PathVariable Long libroId) {
52         return libros.findById(libroId).orElseThrow(() ->
53             new RecursoNoEncontradoException("El libro con id " + libroId + " no
54                 existe"));
55     }
56
57     @PostMapping
58     public ResponseEntity<Libro> crear(@Valid @RequestBody LibroInput in) {
59         Libro guardado = libros.save(new Libro(in.titulo(), in.autor(),
60             in.isbn(), in.categoria(), in.ejemplaresDisponibles()));
61         return ResponseEntity
62             .created(URI.create("/api/v1/libros/" + guardado.getId())) // 201
63             .body(guardado);
64     }
65
66     @DeleteMapping("/{libroId}")
67     public ResponseEntity<Void> eliminar(@PathVariable Long libroId) {
68         if (!libros.existsById(libroId)) {
69             throw new RecursoNoEncontradoException("El libro con id " + libroId + "
70                 no existe");
71         }
72         libros.deleteById(libroId);
73         return ResponseEntity.noContent().build(); // 204
74     }
75 }
76
77 // ===== archivo: controller/PrestamoController.java =====

```

```

75 package com.uteq.bibliotech.controller;
76
77 import com.uteq.bibliotech.dto.PrestamoInput;
78 import com.uteq.bibliotech.model.*;
79 import com.uteq.bibliotech.repository.*;
80 import jakarta.validation.Valid;
81 import lombok.RequiredArgsConstructor;
82 import org.springframework.http.ResponseEntity;
83 import org.springframework.transaction.annotation.Transactional;
84 import org.springframework.web.bind.annotation.*;
85 import java.net.URI;
86
87 @RestController
88 @RequestMapping("/api/v1/prestamos")
89 @RequiredArgsConstructor
90 public class PrestamoController {
91
92     private final PrestamoRepository prestamos;
93     private final LibroRepository libros;
94     private final SocioRepository socios;
95
96     @PostMapping
97     @Transactional
98     public ResponseEntity<Prestamo> crear(@Valid @RequestBody PrestamoInput in) {
99         Libro libro = libros.findById(in.libroId()).orElseThrow(() ->
100             new RecursoNoEncontradoException("El libro con id " + in.libroId() + "
              no existe"));
101         if (!socios.existsById(in.socioId())) {
102             throw new RecursoNoEncontradoException("El socio con id " + in.socioId
              () + " no existe");
103         }
104         if (libro.getEjemplaresDisponibles() <= 0) {
105             throw new ReglaNegocioException("No hay ejemplares disponibles del
              libro " + libro.getId());
106         }
107         libro.setEjemplaresDisponibles(libro.getEjemplaresDisponibles() - 1);
108         libros.save(libro); // UPDATE libros ...
109         Prestamo p = prestamos.save(new Prestamo(in.libroId(), in.socioId(), in.
              fechaLimite()));
110         return ResponseEntity.created(URI.create("/api/v1/prestamos/" + p.getId()))
              .body(p);
111     }
112
113     @PatchMapping("/{prestamoId}/devolucion")
114     @Transactional
115     public Prestamo devolver(@PathVariable Long prestamoId) {
116         Prestamo p = prestamos.findById(prestamoId).orElseThrow(() ->
117             new RecursoNoEncontradoException("El prestamo con id " + prestamoId + "
              no existe"));
118         if (p.getEstado() == EstadoPrestamo.DEVUELTO) {
119             throw new ReglaNegocioException("El prestamo " + prestamoId + " ya fue
              devuelto");
120         }
121         p.setEstado(EstadoPrestamo.DEVUELTO);
122         libros.findById(p.getLibroId()).ifPresent(l -> {
123             l.setEjemplaresDisponibles(l.getEjemplaresDisponibles() + 1);
124             libros.save(l);
125         });
126         return prestamos.save(p); // UPDATE prestamos
              SET estado...
127     }
128 }

```

El manejador global garantiza que **todos** los errores salgan con el mismo esquema **Error** declarado en el contrato OpenAPI:

Listing 15: controller/GlobalExceptionHandler.java — errores con el esquema uniforme del contrato

```

1 package com.uteq.bibliotech.controller;
2
3 import com.uteq.bibliotech.dto.ApiError;
4 import jakarta.servlet.http.HttpServletRequest;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.authentication.BadCredentialsException;
8 import org.springframework.web.bind.MethodArgumentNotValidException;
9 import org.springframework.web.bind.annotation.ExceptionHandler;
10 import org.springframework.web.bind.annotation.RestControllerAdvice;
11 import java.time.Instant;
12
13 @RestControllerAdvice
14 public class GlobalExceptionHandler {
15
16     private ResponseEntity<ApiError> construir(HttpStatus status, String mensaje,
17                                             HttpServletRequest req) {
18         return ResponseEntity.status(status).body(new ApiError(
19             status.value(), status.getReasonPhrase(), mensaje,
20             Instant.now().toString(), req.getRequestURI()));
21     }
22
23     @ExceptionHandler(RecursoNoEncontradoException.class) // 404
24     public ResponseEntity<ApiError> noEncontrado(RecursoNoEncontradoException e,
25                                             HttpServletRequest req) {
26         return construir(HttpStatus.NOT_FOUND, e.getMessage(), req);
27     }
28
29     @ExceptionHandler({ReglaNegocioException.class,
30                       MethodArgumentNotValidException.class}) // 400
31     public ResponseEntity<ApiError> solicitudInvalida(Exception e,
32                                             HttpServletRequest req) {
33         return construir(HttpStatus.BAD_REQUEST, e.getMessage(), req);
34     }
35
36     @ExceptionHandler(BadCredentialsException.class) // 401
37     public ResponseEntity<ApiError> credenciales(BadCredentialsException e,
38                                             HttpServletRequest req) {
39         return construir(HttpStatus.UNAUTHORIZED, e.getMessage(), req);
40     }
41
42     @ExceptionHandler(Exception.class) // 500
43     public ResponseEntity<ApiError> interno(Exception e, HttpServletRequest req) {
44         return construir(HttpStatus.INTERNAL_SERVER_ERROR,
45             "Error interno del servidor", req);
46     }
47 }

```

Listing 16: BibliotechApplication.java — clase principal con main() completo

```

1 package com.uteq.bibliotech;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class BibliotechApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(BibliotechApplication.class, args);
11     }
12 }

```

3.10. Tabla de clases por paquete

Cuadro 4: Atribución de clases a paquetes del proyecto `com.uteq.bibliotech`

Paquete	Clase / Interfaz
(raíz)	BibliotechApplication
model	Libro, Prestamo, Socio, Usuario, EstadoPrestamo
repository	LibroRepository, PrestamoRepository, SocioRepository, UsuarioRepository
security	JwtService, JwtAuthFilter, SecurityConfig
dto	LoginRequest, TokenResponse, LibroInput, PrestamoInput, PaginaResponse, ApiError
controller	AuthController, LibroController, PrestamoController, GlobalExceptionHandler, excepciones propias

3.11. Compilar, ejecutar y verificar de extremo a extremo

Secuencia de arranque y prueba

```
# 1) Base de datos (crea/arranca el contenedor en segundo plano)
docker compose up -d db

# 2) Backend (descarga dependencias, compila y arranca en :8080)
mvn spring-boot:run

# 3) Obtener el JWT (el token "nace" aqui)
curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"maria.perez@uteq.edu.ec","password":"Secreto123!"}'

# 4) Consumir un endpoint protegido con el token recibido
curl -s http://localhost:8080/api/v1/libros \
  -H "Authorization: Bearer <pegar-token-aqui>"

# 5) Verificar en la base de datos que un POST realmente persiste:
# "exec" entra al contenedor en ejecucion; "-c" ejecuta una unica
# consulta SQL en psql y devuelve el resultado sin abrir sesion.
docker exec -it bibliotech-db psql -U biblio -d bibliotech \
  -c "SELECT id, estado FROM prestamos;"
```

Con esta infraestructura funcionando, las respuestas que se guardan como ejemplos en la colección Postman del Paso 6 son **evidencia real** extraída de PostgreSQL, no datos inventados.

4. Paso 1 — Identificar recursos y diseñar las URI

Paso 1 — Del dominio a los recursos

1. Listar los **sustantivos** del dominio: *libro*, *préstamo*, *socio*, *credencial*.
2. Convertir cada sustantivo relevante en un **recurso** identificado por una URI (Identificador Uniforme de Recursos) en **plural y minúsculas**: `/libros`, `/prestamos`.
3. Las **acciones** no van en la URI: las expresa el **verbo HTTP** (Protocolo de Transferencia de Hipertexto). Está prohibido `/obtenerLibros` o `/crearPrestamo`.
4. Las relaciones se expresan con **jerarquía o sub-recursos**: la devolución es un estado del préstamo, por eso se modela como `PATCH /prestamos/{prestamoId}/devolucion`.

- Definir el **prefijo de versión** del API: `/api/v1`. Así, un cambio incompatible futuro se publica como `/api/v2` sin romper clientes existentes.

Cuadro 5: Decisiones de diseño de URI en BiblioTech (correcto vs. incorrecto)

Incorrecto (estilo RPC)	Correcto (estilo REST)	Justificación
GET <code>/obtenerLibros</code>	GET <code>/libros</code>	Sustantivo, no verbo
POST <code>/libro/crear</code>	POST <code>/libros</code>	El verbo lo da HTTP
GET <code>/libros?id=101</code>	GET <code>/libros/101</code>	Identidad en la ruta
POST <code>/devolverPrestamo</code>	PATCH <code>/prestamos/7001/devolucion</code>	Cambio parcial de estado

5. Paso 2 — Tabla de endpoints (mínimo 5)

Antes de abrir Swagger Editor, el equipo completa esta tabla de planificación. Es el insumo directo del YAML y de la colección Postman.

Cuadro 6: Endpoints del microservicio BiblioTech (7 operaciones sobre 5 rutas)

Método	Ruta	Entrada	Respuestas
POST	<code>/auth/login</code>	Body: email, password	200 token JWT, 400, 401, 500
GET	<code>/libros</code>	Query: page, size, categoria	200 lista paginada, 400, 500
POST	<code>/libros</code>	Body: LibroInput	201 Libro, 400, 500
GET	<code>/libros/{libroId}</code>	Path: libroId	200 Libro, 404, 500
DELETE	<code>/libros/{libroId}</code>	Path: libroId	204 sin contenido, 404, 500
POST	<code>/prestamos</code>	Body: PrestamoInput	201 Prestamo, 400, 404, 500
PATCH	<code>/prestamos/{id}/devolucion</code>	Path: prestamoId	200 Prestamo, 400, 404, 500

Por qué estos códigos

200 lectura/actualización exitosa; 201 recurso creado; 204 eliminado sin cuerpo; 400 validación fallida; 401 sin token o token inválido; 404 el recurso no existe; 500 fallo interno. La rúbrica exige como mínimo 200, 400, 404 y 500 en cada endpoint; los demás suman precisión semántica.

6. Paso 3 — Escribir el contrato OpenAPI 3.0 en Swagger Editor

Procedimiento en Swagger Editor

- Abrir <https://editor.swagger.io> en el navegador. Borrar el ejemplo *Petstore* precargado (Ctrl+A, Supr).
- Escribir el contrato **en este orden**: bloque `info` → `servers` → `security` global → `paths` → `components` (esquemas, respuestas reutilizables y `securitySchemes`).
- El panel derecho renderiza la documentación en tiempo real; el panel izquierdo marca en rojo cualquier error de sintaxis YAML o de la especificación.
- Definir primero los **esquemas de datos** en `components/schemas` y referenciarlos con `$ref` (nunca duplicar estructuras en cada endpoint).
- Declarar la autenticación **Bearer JWT** una sola vez en `components/securitySchemes` y aplicarla globalmente con el bloque `security` raíz; el endpoint `/auth/login` la desactiva

localmente con `security: []`.

El contrato completo y validado del caso BiblioTech es el siguiente (archivo `openapi.yaml`, adjunto también como archivo independiente):

Listing 17: Contrato OpenAPI 3.0 completo — BiblioTech API (validado con `yaml.safe_load` y Swagger Editor)

```

1 openapi: 3.0.3
2 info:
3   title: BiblioTech API
4   version: 1.0.0
5   description: >
6     API REST del microservicio de gestion de libros y prestamos
7     de la Biblioteca Universitaria BiblioTech. Permite consultar
8     el catalogo, registrar libros, crear prestamos y registrar
9     devoluciones. Todos los endpoints (excepto /auth/login)
10    requieren autenticacion Bearer JWT.
11   contact:
12     name: Equipo BiblioTech
13     email: bibliotech@uteq.edu.ec
14
15   servers:
16     - url: http://localhost:8080/api/v1
17       description: Servidor de desarrollo local
18
19   security:
20     - bearerAuth: []
21
22   tags:
23     - name: Autenticacion
24       description: Emision de tokens JWT
25     - name: Libros
26       description: Gestion del catalogo de libros
27     - name: Prestamos
28       description: Gestion de prestamos y devoluciones
29
30   paths:
31     /auth/login:
32       post:
33         tags: [Autenticacion]
34         summary: Autenticar usuario y obtener token JWT
35         operationId: login
36         security: []
37         requestBody:
38           required: true
39           content:
40             application/json:
41               schema:
42                 $ref: '#/components/schemas/CredencialesInput'
43         responses:
44           '200':
45             description: Autenticacion exitosa
46             content:
47               application/json:
48                 schema:
49                   $ref: '#/components/schemas/TokenResponse'
50           '400':
51             $ref: '#/components/responses/BadRequest'
52           '401':
53             description: Credenciales invalidas
54             content:
55               application/json:
56                 schema:

```



```

57         $ref: '#/components/schemas/Error'
58     '500':
59         $ref: '#/components/responses/ServerError'
60
61 /libros:
62     get:
63         tags: [Libros]
64         summary: Listar libros del catalogo (con paginacion y filtro)
65         operationId: listarLibros
66         parameters:
67             - name: page
68               in: query
69               description: Numero de pagina (inicia en 0)
70               schema:
71                 type: integer
72                 minimum: 0
73                 default: 0
74             - name: size
75               in: query
76               description: Cantidad de elementos por pagina
77               schema:
78                 type: integer
79                 minimum: 1
80                 maximum: 100
81                 default: 20
82             - name: categoria
83               in: query
84               description: Filtrar por categoria del libro
85               schema:
86                 type: string
87         responses:
88             '200':
89                 description: Lista paginada de libros
90                 content:
91                     application/json:
92                         schema:
93                             $ref: '#/components/schemas/LibroPaginado'
94             '400':
95                 $ref: '#/components/responses/BadRequest'
96             '500':
97                 $ref: '#/components/responses/ServerError'
98     post:
99         tags: [Libros]
100         summary: Registrar un nuevo libro en el catalogo
101         operationId: crearLibro
102         requestBody:
103             required: true
104             content:
105                 application/json:
106                     schema:
107                         $ref: '#/components/schemas/LibroInput'
108         responses:
109             '201':
110                 description: Libro creado correctamente
111                 content:
112                     application/json:
113                         schema:
114                             $ref: '#/components/schemas/Libro'
115             '400':
116                 $ref: '#/components/responses/BadRequest'
117             '500':
118                 $ref: '#/components/responses/ServerError'
119
120 /libros/{libroId}:
121     get:

```

```

122     tags: [Libros]
123     summary: Obtener un libro por su identificador
124     operationId: obtenerLibro
125     parameters:
126     - $ref: '#/components/parameters/LibroId'
127     responses:
128     '200':
129         description: Libro encontrado
130         content:
131             application/json:
132                 schema:
133                     $ref: '#/components/schemas/Libro'
134     '404':
135         $ref: '#/components/responses/NotFound'
136     '500':
137         $ref: '#/components/responses/ServerError'
138     delete:
139     tags: [Libros]
140     summary: Eliminar un libro del catalogo
141     operationId: eliminarLibro
142     parameters:
143     - $ref: '#/components/parameters/LibroId'
144     responses:
145     '204':
146         description: Libro eliminado (sin contenido)
147     '404':
148         $ref: '#/components/responses/NotFound'
149     '500':
150         $ref: '#/components/responses/ServerError'
151
152 /prestamos:
153     post:
154     tags: [Prestamos]
155     summary: Crear un prestamo de libro a un socio
156     operationId: crearPrestamo
157     requestBody:
158         required: true
159         content:
160             application/json:
161                 schema:
162                     $ref: '#/components/schemas/PrestamoInput'
163     responses:
164     '201':
165         description: Prestamo registrado
166         content:
167             application/json:
168                 schema:
169                     $ref: '#/components/schemas/Prestamo'
170     '400':
171         $ref: '#/components/responses/BadRequest'
172     '404':
173         $ref: '#/components/responses/NotFound'
174     '500':
175         $ref: '#/components/responses/ServerError'
176
177 /prestamos/{prestamoId}/devolucion:
178     patch:
179     tags: [Prestamos]
180     summary: Registrar la devolucion de un prestamo
181     operationId: registrarDevolucion
182     parameters:
183     - name: prestamoId
184       in: path
185       required: true
186     description: Identificador del prestamo

```

```

187     schema:
188         type: integer
189         format: int64
190     responses:
191         '200':
192             description: Devolucion registrada
193             content:
194                 application/json:
195                     schema:
196                         $ref: '#/components/schemas/Prestamo'
197         '400':
198             $ref: '#/components/responses/BadRequest'
199         '404':
200             $ref: '#/components/responses/NotFound'
201         '500':
202             $ref: '#/components/responses/ServerError'
203
204 components:
205     securitySchemes:
206         bearerAuth:
207             type: http
208             scheme: bearer
209             bearerFormat: JWT
210             description: Token JWT emitido por POST /auth/login
211
212     parameters:
213         LibroId:
214             name: libroId
215             in: path
216             required: true
217             description: Identificador unico del libro
218             schema:
219                 type: integer
220                 format: int64
221
222     responses:
223         BadRequest:
224             description: Solicitud invalida (validacion fallida)
225             content:
226                 application/json:
227                     schema:
228                         $ref: '#/components/schemas/Error'
229         NotFound:
230             description: Recurso no encontrado
231             content:
232                 application/json:
233                     schema:
234                         $ref: '#/components/schemas/Error'
235         ServerError:
236             description: Error interno del servidor
237             content:
238                 application/json:
239                     schema:
240                         $ref: '#/components/schemas/Error'
241
242     schemas:
243         CredencialesInput:
244             type: object
245             required: [email, password]
246             properties:
247                 email:
248                     type: string
249                     format: email
250                     example: maria.perez@uteq.edu.ec
251                 password:

```

```

252         type: string
253         format: password
254         minLength: 8
255         example: Secreto123!
256
257     TokenResponse:
258         type: object
259         properties:
260             token:
261                 type: string
262                 description: Token JWT con expiracion de 1 hora
263                 example: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
264             tipo:
265                 type: string
266                 example: Bearer
267             expiraEn:
268                 type: integer
269                 description: Segundos hasta la expiracion
270                 example: 3600
271
272     Libro:
273         type: object
274         properties:
275             id:
276                 type: integer
277                 format: int64
278                 example: 101
279             titulo:
280                 type: string
281                 example: Building Microservices
282             autor:
283                 type: string
284                 example: Sam Newman
285             isbn:
286                 type: string
287                 pattern: '^[0-9-]{10,17}$'
288                 example: 978-1-4920-3402-5
289             categoria:
290                 type: string
291                 example: Ingenieria de Software
292             ejemplaresDisponibles:
293                 type: integer
294                 minimum: 0
295                 example: 4
296
297     LibroInput:
298         type: object
299         required: [titulo, autor, isbn, categoria, ejemplaresDisponibles]
300         properties:
301             titulo:
302                 type: string
303                 minLength: 1
304             autor:
305                 type: string
306                 minLength: 1
307             isbn:
308                 type: string
309                 pattern: '^[0-9-]{10,17}$'
310             categoria:
311                 type: string
312             ejemplaresDisponibles:
313                 type: integer
314                 minimum: 0
315
316     LibroPaginado:

```

```
317     type: object
318     properties:
319         contenido:
320             type: array
321             items:
322                 $ref: '#/components/schemas/Libro'
323         pagina:
324             type: integer
325             example: 0
326         totalPaginas:
327             type: integer
328             example: 5
329         totalElementos:
330             type: integer
331             example: 93
332
333     PrestamoInput:
334         type: object
335         required: [libroId, socioId, fechaLimite]
336         properties:
337             libroId:
338                 type: integer
339                 format: int64
340                 example: 101
341             socioId:
342                 type: integer
343                 format: int64
344                 example: 2045
345             fechaLimite:
346                 type: string
347                 format: date
348                 example: '2026-06-30'
349
350     Prestamo:
351         type: object
352         properties:
353             id:
354                 type: integer
355                 format: int64
356                 example: 7001
357             libroId:
358                 type: integer
359                 format: int64
360                 example: 101
361             socioId:
362                 type: integer
363                 format: int64
364                 example: 2045
365             fechaPrestamo:
366                 type: string
367                 format: date
368                 example: '2026-06-16'
369             fechaLimite:
370                 type: string
371                 format: date
372                 example: '2026-06-30'
373             estado:
374                 type: string
375                 enum: [ACTIVO, DEVUELTO, VENCIDO]
376                 example: ACTIVO
377
378     Error:
379         type: object
380         properties:
381             status:
```

```

382     type: integer
383     example: 404
384   error:
385     type: string
386     example: Not Found
387   message:
388     type: string
389     example: El libro con id 999 no existe
390   timestamp:
391     type: string
392     format: date-time
393     example: '2026-06-16T10:30:00Z'
394   path:
395     type: string
396     example: /api/v1/libros/999

```

Errores que más se penalizan en este paso

- Sangría YAML inconsistente (mezclar 2 y 4 espacios, o usar tabuladores): el archivo no parsea.
- Tipos incorrectos en los esquemas (por ejemplo, `id` como `string` cuando la base de datos usa enteros).
- Omitir las respuestas de error (400, 404, 500) o declararlas sin esquema `Error`.
- Definir `bearerAuth` pero no aplicarlo (falta el bloque `security` raíz).
- Copiar el contrato de otro equipo cambiando solo los nombres: se detecta por estructura idéntica y constituye falta de integridad académica.

7. Paso 4 — Validar el contrato

Lista de validación obligatoria

1. En Swagger Editor: el panel izquierdo debe mostrar **cero errores** (sin marcas rojas) y el derecho debe renderizar las tres etiquetas (*Autenticacion*, *Libros*, *Prestamos*) con el candado de seguridad visible en cada operación protegida.
2. Probar el botón **Try it out** de `GET /libros`: el editor genera el `curl` equivalente; verificar que incluye el encabezado `Authorization`.
3. Validación local por línea de comandos (opcional pero recomendada):

```

# Validacion sintactica YAML
python3 -c "import yaml; yaml.safe_load(open('openapi.yaml')); print('YAML
    valido')"

# Validacion completa contra la especificacion OpenAPI (Node.js)
npx @redocly/cli lint openapi.yaml

```

8. Paso 5 — Exportar y subir al repositorio GitHub

Exportación y versionado

1. En Swagger Editor: menú **File** → **Save as YAML**. Renombrar el archivo descargado a `openapi.yaml`.
2. Dentro del repositorio del equipo, crear la carpeta exacta `/docs/api/` y confirmar los cambios:

```
cd proyecto-pfc
```

```
mkdir -p docs/api
mv ~/Descargas/openapi.yaml docs/api/openapi.yaml
git add docs/api/openapi.yaml
git commit -m "docs(api): contrato OpenAPI 3.0 del microservicio principal"
git push origin main
```

3. Verificar en el navegador que el archivo es visible en GitHub y que el repositorio es **público** o con acceso concedido al docente.

Advertencia de evaluación

Un repositorio privado o inexistente (error 404) impide verificar la evidencia y se penaliza de forma severa según las reglas del curso. Comprobar el acceso **desde una ventana de incógnito** antes de la entrega.

9. Paso 6 — Colección Postman con ejemplos

Construcción de la colección

1. Abrir Postman → **Import** → pestaña **File** → seleccionar `openapi.yaml`. Postman genera automáticamente una colección con las 7 operaciones organizadas por etiqueta. (Alternativa manual: **New Collection** y crear cada petición.)
2. Crear un **Environment** llamado **BiblioTech-Local** con dos variables: `baseUrl = http://localhost:8080/api/v1` y `token` (vacía). Todas las URL de la colección deben usar `{{baseUrl}}`.
3. En la pestaña **Authorization** de la colección (nivel raíz), seleccionar tipo **Bearer Token** con valor `{{token}}`; cada petición hereda la autenticación (*Inherit auth from parent*). En `POST /auth/login`, fijar tipo **No Auth**.
4. Automatizar la captura del token con un *script* en la pestaña **Scripts (Post-response)** de la petición de login:

```
const data = pm.response.json();
pm.environment.set("token", data.token);
```

5. Para cada una de las 7 peticiones, ejecutarla (o simular el cuerpo) y guardar un **ejemplo**: botón **Save Response** → **Save as example**. Cada petición debe tener al menos un ejemplo de **request** con cuerpo JSON realista y un ejemplo de **response** exitosa; se recomienda añadir además un ejemplo de error (404 o 400).

Ejemplo de cuerpo de petición y de respuesta que deben quedar guardados para `POST /prestamos`:

Listing 18: Ejemplo request (izquierda conceptual) y response 201 (derecha conceptual) de `POST /prestamos`

<pre>// Request body { "libroId": 101, "socioId": 2045, "fechaLimite": "2026-06-30" }</pre>	<pre>// Response 201 Created { "id": 7001, "libroId": 101, "socioId": 2045, "fechaPrestamo": "2026-06-16", "fechaLimite": "2026-06-30", "estado": "ACTIVO" }</pre>
---	--

Exportar y subir la colección

1. Clic derecho sobre la colección → **Export** → formato **Collection v2.1** → guardar como `coleccion-postman.json`.
2. Subirla a la misma carpeta del repositorio:

```
mv ~/Descargas/coleccion-postman.json docs/api/
git add docs/api/coleccion-postman.json
git commit -m "docs(api): coleccion Postman con ejemplos de request/
response"
git push origin main
```

10. Lista de verificación de entrega

Cuadro 7: Lista de verificación antes de cerrar el taller

Requisito verificable	Cumple
info con título, versión y descripción del API	☐
Mínimo 5 endpoints con método, ruta, parámetros, request body y responses	☐
Respuestas 200, 400, 404 y 500 declaradas con esquema Error	☐
Esquemas en <code>components/schemas</code> con tipos, formatos y required correctos	☐
Esquema de autenticación bearerAuth (HTTP Bearer, formato JWT) aplicado globalmente	☐
Contrato sin errores en Swagger Editor	☐
<code>/docs/api/openapi.yaml</code> visible en GitHub (repositorio accesible)	☐
Colección Postman v2.1 con las operaciones y ejemplos de request/response	☐
<code>/docs/api/coleccion-postman.json</code> visible en GitHub	☐
Declaración de uso de inteligencia artificial generativa incluida en la entrega	☐

11. Adicional — De una API REST a una API RESTful, paso a paso

El término **RESTful** designa una API que **cumple plenamente** las restricciones del estilo arquitectónico REST definido por Fielding [1], y no solo “usa HTTP con JSON”. La referencia práctica para medir ese cumplimiento es el **Modelo de Madurez de Richardson** [2], con cuatro niveles:

Cuadro 8: Modelo de Madurez de Richardson aplicado a BiblioTech

Nivel	Nombre	Qué significa en la práctica
0	El pantano de POX	Un solo endpoint (p. ej. <code>POST /api</code>) que recibe “órdenes” en el cuerpo. Es RPC sobre HTTP.
1	Recursos	URI distintas por recurso (<code>/libros</code> , <code>/prestamos</code>), pero todo con POST.
2	Verbos HTTP + códigos de estado	GET/POST/PUT/PATCH/DELETE con semántica correcta y códigos 2xx/4xx/5xx precisos. La mayoría de las API “REST” se quedan aquí.
3	HATEOAS	Las respuestas incluyen hipervínculos que indican al cliente qué acciones puede hacer a continuación. Solo aquí la API es plenamente RESTful.

A continuación, la **secuencia de transformación** que el equipo debe aplicar a su API:

Transformación 1 — Recursos con sustantivos (Nivel 0 → 1)

Eliminar verbos de las URI y modelar cada entidad como recurso plural. Antes: `POST /api {accion: "buscarLibro"}`. Después: `GET /libros/101`. Verificación: ninguna URI del contrato contiene un verbo.

Transformación 2 — Semántica de los verbos HTTP (Nivel 1 → 2)

Asignar cada operación al verbo correcto respetando dos propiedades formales definidas en la semántica de HTTP [3]:

- **Seguro** (*safe*): no modifica estado del servidor.
- **Idempotente**: repetir la misma petición n veces produce el mismo estado que ejecutarla una vez.

Verbo	Seguro	Idempotente	Uso en BiblioTech
GET	Sí	Sí	Leer libros y préstamos
POST	No	No	Crear libro, crear préstamo, login
PUT	No	Sí	Reemplazo completo de un libro
PATCH	No	No*	Registrar devolución (cambio parcial)
DELETE	No	Sí	Eliminar libro

*PATCH no es idempotente por definición, aunque una implementación concreta puede serlo.

Transformación 3 — Códigos de estado precisos (consolida el Nivel 2)

Prohibido responder 200 OK con un mensaje de error en el cuerpo. Cada situación tiene su código: creación → 201 con encabezado `Location: /libros/102`; eliminación → 204; validación fallida → 400; sin token → 401; token válido sin permisos → 403; inexistente → 404; conflicto de negocio (libro sin ejemplares disponibles) → 409; fallo interno → 500.

Transformación 4 — Comunicación sin estado (restricción *stateless*)

El servidor **no guarda sesiones**: cada petición lleva toda la información necesaria para autenticarla y procesarla. En BiblioTech esto se cumple porque el JWT [4] viaja en cada petición (`Authorization: Bearer <token>`) y contiene la identidad y la expiración firmadas; cualquier réplica del microservicio puede atender cualquier petición, lo que habilita el escalado horizontal. Verificación: la API no usa cookies de sesión ni almacenamiento de sesión en el servidor.

Transformación 5 — Representaciones y negociación de contenido

El recurso es un concepto; lo que viaja es una **representación** (JSON por defecto). El cliente la negocia con encabezados: `Accept: application/json` en la petición y `Content-Type: application/json` en la respuesta. En OpenAPI esto ya quedó declarado en cada bloque `content`.

Transformación 6 — HATEOAS (Nivel 2 → 3, plenamente RESTful)

HATEOAS (*Hypermedia As The Engine Of Application State*, hipermedios como motor del estado de la aplicación) significa que cada respuesta incluye los **enlaces** a las acciones válidas según el estado actual del recurso. El cliente ya no necesita conocer las rutas de memoria: las descubre. Respuesta de un préstamo ACTIVO con HATEOAS:

```
{
  "id": 7001,
```

```

"libroId": 101,
"socioId": 2045,
"estado": "ACTIVO",
"fechaLimite": "2026-06-30",
"_links": {
  "self": { "href": "/api/v1/prestamos/7001" },
  "devolucion": { "href": "/api/v1/prestamos/7001/devolucion", "method": "PATCH" },
  "libro": { "href": "/api/v1/libros/101" }
}
}

```

Si el préstamo ya estuviera DEVUELTO, el enlace **devolucion** **desaparece**: los enlaces representan las transiciones de estado permitidas. En Spring Boot 3.x esto se implementa con el módulo `spring-boot-starter-hateoas` (clases `EntityModel` y `WebMvcLinkBuilder`).

Transformación 7 — Buenas prácticas complementarias del nivel profesional

- **Versionado** en la URI (`/api/v1`) para evolucionar sin romper clientes.
- **Paginación, filtrado y ordenamiento** en colecciones: `GET /libros?page=0&size=20&categoria=redes` (ya incluido en el contrato).
- **Cacheo** de lecturas: encabezados `ETag` y `Cache-Control` en GET, aprovechando la restricción `cacheable` de REST.
- **Errores uniformes**: un único esquema `Error` (status, error, message, timestamp, path) en toda la API.
- **Nombres consistentes**: rutas en minúsculas y plural; propiedades JSON en *camelCase* [6].

Criterio de autoevaluación final

La API del equipo es **RESTful** cuando: (1) toda URI es un sustantivo, (2) los verbos HTTP respetan su semántica de seguridad e idempotencia, (3) los códigos de estado son precisos, (4) la comunicación es sin estado vía JWT, (5) las representaciones se negocian por encabezados y (6) las respuestas exponen enlaces HATEOAS. Los puntos (1)–(5) son obligatorios para PE-U3/PFC E2; el punto (6) es el diferenciador de excelencia.

12. Cómo replicar este taller con el PFC propio

Cada equipo repite los pasos 1 a 6 sustituyendo los recursos de BiblioTech por los del microservicio principal de su PFC. Mapeo sugerido (el equipo puede ajustarlo con justificación):

Cuadro 9: Mapeo del ejemplo BiblioTech a los recursos de cada PFC

Equipo / PFC	Recurso $\approx$ /libros	Recurso $\approx$ /prestamos	Sub-recurso de estado $\approx$ /devolucion
TiendaTech (e-commerce)	/productos	/pedidos	/pedidos/{id}/pago
SGA Escuela Provincias Unidas	/estudiantes	/matriculas	/matriculas/{id}/retiro
Laboratorios Informáticos	/laboratorios	/reservas	/reservas/{id}/cierre
Soporte Técnico ISP	/clientes	/tickets	/tickets/{id}/resolucion

En todos los casos se conservan sin cambios: el endpoint `POST /auth/login`, el esquema `Error`, el esquema de paginación, la seguridad `bearerAuth` global y la estructura de carpetas `/docs/api/` del repositorio.

Referencias

Referencias

- [1] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.
- [2] M. Fowler, “Richardson Maturity Model: steps toward the glory of REST,” martinowler.com, Mar. 2010. [En línea]. Disponible: <https://martinfowler.com/articles/richardsonMaturityModel.html>
- [3] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” Internet Engineering Task Force, RFC 9110, Jun. 2022. doi: 10.17487/RFC9110.
- [4] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015. doi: 10.17487/RFC7519.
- [5] OpenAPI Initiative, “OpenAPI Specification v3.0.3,” Feb. 2020. [En línea]. Disponible: <https://spec.openapis.org/oas/v3.0.3>
- [6] M. Masse, *REST API Design Rulebook*. Sebastopol, CA, USA: O’Reilly Media, 2011. ISBN: 978-1-449-31050-9.